MOBILE APPLICATION DEVELOPMENT

BSC (HONS) IN INFORMATION TECHNOLOGY

iOS Application Development Overview

iOS Architecture, Swift & SwiftUI, the App Store Ecosystem & a Comparison with Android



Learning Outcomes

01 Describe the iOS ecosystem and the control Apple holds over hardware, software, and distribution.

02 Explain the four layers of the iOS architecture and what each layer provides.

03 Read basic Swift syntax: variables, constants, optionals, functions, and classes.

04 Distinguish UIKit with Storyboards from the declarative SwiftUI approach.

05 Outline how an iOS app reaches users through the App Store review process.

06 Compare iOS and Android across language, tooling, UI framework, and distribution.

What Is the iOS Ecosystem?

- iOS is a closed, tightly controlled ecosystem owned and managed end to end by Apple.
- Apple supplies the hardware, the operating system, the tools, the language, and the store.
- This topic is covered in theory only, since iOS development requires macOS hardware.

Apple controls

Devices, iOS itself, Xcode, Swift, the UI frameworks, and the App Store.

Android differs

Many manufacturers, an open-source OS, and several competing app stores.

Consequence

Far fewer variations to support, but much less freedom for the developer.

Apple Hardware and Devices

- iOS apps run only on Apple hardware: iPhone, iPad, and iPod Touch.
- Apple designs both the silicon and the operating system, so the two are tightly optimised.
- Screen sizes, chipsets, and OS versions form a small and predictable set.

iOS

A short list of current devices, most of them running a recent iOS version.

Android

Thousands of models from many vendors, spread across many OS versions.

Less fragmentation means simpler testing and more consistent UI, but no low-cost device tier.

iOS Layered Architecture

Cocoa Touch

The top layer, where developers work: UIKit, SwiftUI, gestures, and notifications.

Media

Graphics, audio, video, and animation, via Core Animation.

Core Services

Foundation types, networking, location, and persistence such as Core Data.

Core OS

The lowest layer: memory, security, threading, and file system access.

- Each layer builds on the one below it and hides that complexity from the layer above.
- Application developers spend nearly all of their time in the Cocoa Touch layer.

Xcode and the Developer Toolchain

Xcode

Apple’s official IDE for iOS, iPadOS, macOS, watchOS, and tvOS. Runs only on macOS.

Interface Builder

Visual editor for Storyboards, and the live preview canvas for SwiftUI.

iOS Simulator

Runs apps on a simulated iPhone or iPad, without any physical device.

Instruments and LLDB

The profiler and the debugger: memory, performance, and breakpoints.

Inside an iOS Project

01 AppDelegate.swift — handles application-level events such as launch and termination.

02 SceneDelegate.swift — manages UI scenes and multi-window support.

03 ViewController.swift — the code that controls one screen.

04 Main.storyboard — the visual layout of screens and the navigation between them.

05 Assets.xcassets — images, icons, and colour sets used by the app.

06 Info.plist — app configuration: name, version, permissions, and capabilities.

AppDelegate and SceneDelegate

```
import UIKit

@main
class AppDelegate: UIResponder,
                   UIApplicationDelegate {
}


class SceneDelegate: UIResponder,
                     UIWindowSceneDelegate {
    var window: UIWindow?
}
```

- AppDelegate handles app launch and termination — much like Android's Application class.
- SceneDelegate controls the lifecycle of one window and decides which ViewController is shown first.

ViewControllers

- A ViewController is the object that controls a single screen of the app.
- It holds the UI logic for that screen and responds to the user's interaction.
- It is the iOS equivalent of an Android Activity.

Owns a screen

One ViewController manages one view hierarchy.

Handles input

Button taps and text entry arrive here as actions.

Has a lifecycle

The system calls methods as the screen appears and disappears.

ViewController Lifecycle

01 viewDidLoad() — called once, after the view loads into memory. Do your setup here.

02 viewWillAppear() — called every time, just before the screen becomes visible.

03 viewDidAppear() — called once the screen is fully visible and animations have finished.

04 viewWillDisappear() — called just before the screen is removed from view.

05 viewDidDisappear() — called after the screen has gone. Stop timers and observers here.

06 Roughly parallel to the Android Activity lifecycle: onCreate, onStart, onResume, onPause, onStop.

Why Swift?

- Swift is Apple’s modern language for all its platforms, and has replaced Objective-C for new work.
- It is designed to be safe, fast, and readable, with strict checking at compile time.
- Whole classes of runtime crash are prevented by the language itself.

Safety

Optionals and strict typing catch missing or wrong values before the app runs.

Clarity

Type inference keeps code short without giving up any type safety.

Kotlin plays the same role on Android: a modern, safer replacement for an older language.

Swift Basics: Variables, Constants, and Types

```
var score = 90              // this value can change
let maxScore = 100          // this value is fixed

let name = "Kamal"          // type inferred as String
let age: Int = 21           // type stated explicitly

func sum(a: Int, b: Int) -> Int { a + b }
```

var

Declares a variable whose value may change later.

let

Declares a constant; the value is fixed once assigned.

Type safety

Swift infers the type where it can, but still checks it strictly.

Optionals: Swift's Answer to Null

```
var email: String?          // may hold a value, or nil

// Optional binding — the safe way to unwrap
if let userEmail = email {
    print(userEmail)
}
// userEmail exists only inside those braces

// Nil-coalescing — supply a default value
let shown = email ?? "guest@site.com"

// Avoid force unwrapping (email!) — it crashes when nil.
```

- An optional may hold a value or nil, and the compiler forces you to handle both cases.
- Swift's defence against the null pointer exception, much like Kotlin's nullable types.

Storyboards and UIKit

- A Storyboard is a visual file in which screens are drawn and navigation is designed.
- UI elements such as UILabel, UITextField, and UIButton are dragged onto the canvas.
- Each element is then connected to code so it can be read from or reacted to.

IBOutlet

Connects a UI element to a property in code.

IBAction

Connects a user action, such as a tap, to a method.

Segue

A visual link that moves the app from one screen to the next.

SwiftUI: Building UI in Code

```
import SwiftUI
struct LoginView: View {

    @State private var username = ""

    var body: some View {
        VStack {
            TextField("Username", text: $username)
                .textFieldStyle(.roundedBorder)

            Button("Login") { print(username) }
        }

        .padding()
    }
}

// The view redraws automatically whenever @State changes.
```

UIKit vs SwiftUI

UIKit

UI built visually in Storyboards.

State is updated by hand in code.

Mature, with years of examples.

SwiftUI

UI written declaratively in Swift.

State changes redraw the UI automatically.

UIKit still runs most shipping apps; SwiftUI is Apple’s direction for new development.

The App Store Ecosystem

- Apps reach users through one channel only: the Apple App Store.
- Publishing requires a paid Apple Developer Account and a digitally signed build.
- Every submission passes an automated and human review before it can be released.

Security

Apps are sandboxed and signed, and are vetted before they reach users.

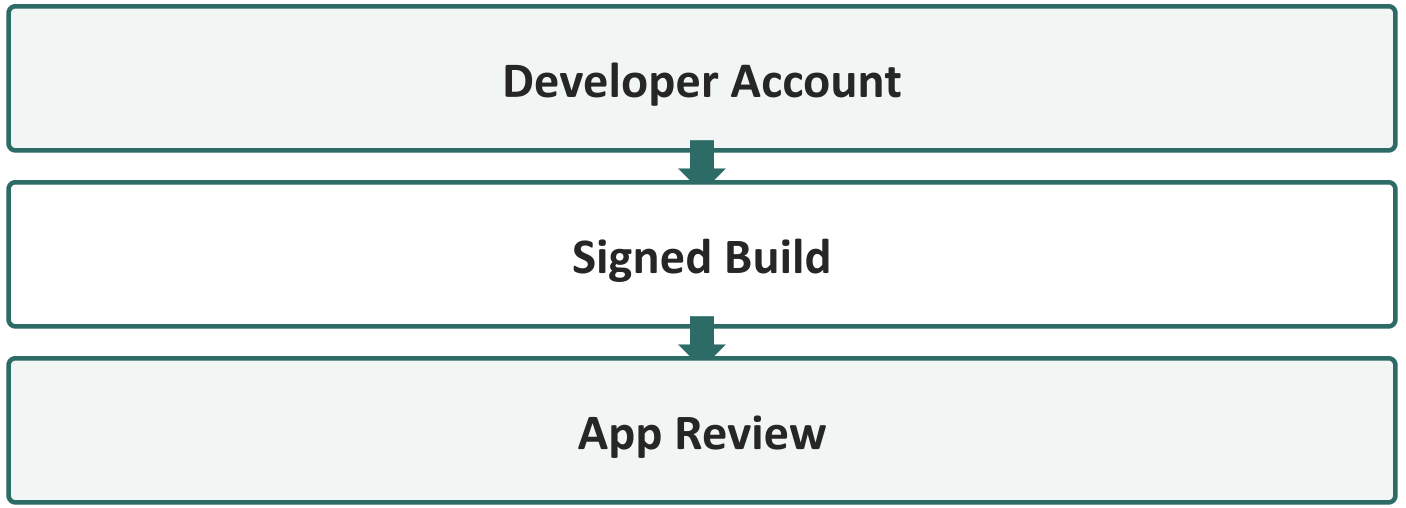
Privacy

Data use must be declared, and the user grants permission at runtime.

Consistency

Apple’s Human Interface Guidelines are enforced during review.

From Code to the App Store



Account cost
“USD 99 / year”

Typical review
“1–3 days”

Outcome
“Approved or rejected”

iOS vs Android: A Comparison

Language

iOS uses Swift; Android uses Kotlin. Both are modern, safe, statically typed languages.

UI framework

iOS uses UIKit or SwiftUI; Android uses XML layouts or Jetpack Compose.

Tooling

Xcode runs on macOS only; Android Studio runs on Windows, macOS, and Linux.

Distribution

iOS ships only through the App Store; Android allows other stores and sideloading.

- Screen unit: iOS uses a ViewController, Android an Activity or Fragment — the same idea, different name.
- The concepts transfer well; what changes is the syntax, the tools, and the level of control.

Key Takeaways

01 iOS is a closed ecosystem: Apple controls the hardware, the OS, the tools, and the only store.

02 The OS is layered — Cocoa Touch, Media, Core Services, Core OS — and you work at the top.

03 Swift is safe and modern; optionals force you to handle missing values explicitly.

04 UIKit builds UI visually with Storyboards; SwiftUI builds it declaratively in code.

05 Publishing needs a developer account and must pass Apple’s review before release.

06 iOS and Android solve the same problems with different tools — the concepts carry across.

Discussion / Quick Check

01 Why is iOS called a closed ecosystem, and what does that mean for a developer?

02 Name the four layers of the iOS architecture. Which one would you work in?

03 What problem do Swift optionals solve, and what is the Android equivalent?

04 Give one difference between building a screen with a Storyboard and with SwiftUI.

05 What is a ViewController, and which Android component matches it?

06 Name two ways distributing an iOS app differs from distributing an Android app.

Thank You

Questions & Discussion